

CPT205 Assessment 2:

3D Modelling Project

Junyan He

Information and Computer Science
School of Advanced Technology
Xi'an Jiaotong-Liverpool University (XJTLU)
Suzhou, China

Email: Junyan.He23@student.xjtlu.edu.cn
Student ID: 2363304



CPT205 – Computer Graphics
Academic Year 2025–2026

I. INTRODUCTION

The project aims to build a small interactive world with a futuristic atmosphere using a narrative structure. The user enters the application through a simple “Play” interface, which introduces the backstory and leads into the Room scene. The room functions as the entry point to the cyber world and therefore remains intentionally minimal, using holographic elements and texture-mapped props to create a technological mood.

Following the on-screen instruction, the user presses the space bar to activate the transition sequence. Two texture-based transitions are used: an screen displayed on the computer, followed by a rotating vortex texture that transports the user into the cyber world.

The cyber city serves as the main scene, continuing the cat model I designed in the 2D assignment, and the cat is fixed at the lower part of the viewport. By shifting the world itself, I let the city streets continuously appear and disappear as the character moves forward. This design is inspired by the looped structure from Lab8’s train system. To present richer world layers, I built multiple subsystems in the cyber city, including a weather system, traffic system, building system, and holographic advertisement system. I also focused on the overall color tone, so different skyboxes and palettes are used for daytime, dusk, and night.

II. README

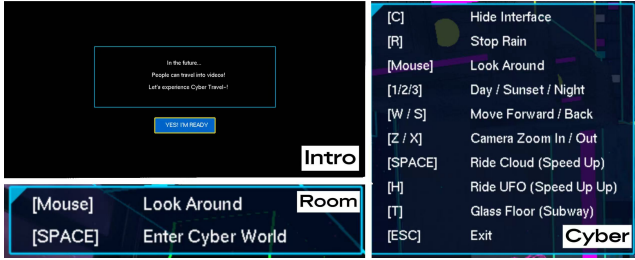


Fig. 1. Overall interaction flow between the Intro, Room and Cyber City scenes.

A. Intro

This section introduces the world setting. The user selects a button to enter the world, and the program first enters the Room scene, as shown in Fig. 1.

B. Scene 1 — Room Scene Controls

The input actions for the Room scene are listed in Table I.

TABLE I
CONTROLS IN ROOM SCENE

Action	Function
Mouse Move	Look around the room (left–right, up–down)
SPACE	Trigger the storyline: screen switches → vortex texture → enter the Cyber world

C. Scene 2 — Cyber City Controls

This is the main scene. During the first 2.5 seconds, a welcome message is shown automatically, followed by the control HUD, as summarized in Fig. 1. Moreover, the full control set for Cyber City is provided in Table II, outlining all available movement, camera, and mode-switch actions.

TABLE II
CONTROLS IN CYBER CITY SCENE

Key / Mouse	Description
Mouse Move	Rotate view (Look Around)
Z / X	Camera zoom in / out
W / S	Move forward / backward (World Scrolling Movement)
1	Switch to Day Mode
2	Switch to Sunset Mode
3	Switch to Night Mode
R	Toggle rain system (Rain / Digital Rain at Night)
T	Toggle glass-floor mode (view underground trains)
SPACE	Toggle Cloud Ride mode
H	Toggle UFO Hyper Warp mode (high-speed flight)
C	Show / hide HUD interface
ESC	Exit the program

III. DESIGN

A. Overall Architecture

The entire program uses a Finite State Machine (FSM) for scene management, covering four main parts: two main worlds (Room and Cyber City) and two transitions. The following sections will explain the design ideas scene by scene, from skyboxes, camera, and lighting to concrete objects and interaction mechanisms.

B. Scene 1: Intro UI (Transition Interface 1)

This interface is used to introduce the world setting, and it is a transition scene that is completely 2D-based. I use flat rectangles to draw the dialogue box and the buttons, without enabling lighting and without any 3D modeling. The purpose of this scene is simply to introduce the world background, as illustrated in Fig. 2.



Fig. 2. Intro UI layout and transition screen combining the initial dialogue interface and the vortex transition.

C. Scene 2: Reality Room (Main Interdace 1)



Fig. 3. Overall layout of the Room scene, including desk, screens, and server towers.

1) *Overall Design and Camera Control*: The Room scene is the “reality space” before entering the cyber world.

In `STATE_ROOM`, the camera is fixed slightly above the desk, and yaw/pitch are updated from mouse input to create a first-person look-around experience, as shown in Fig. 3.

A cool-colored directional light is placed at the upper-right side to simulate night light from outside the window. Its diffuse and ambient components use a blue tone.

2) *Structure of the Room Scene*: The Room scene consists of three main modules, shown in Table III.

TABLE III
MAIN MODULES OF THE ROOM SCENE

Module	Description
A. Room Shell (<code>drawRoom()</code>)	Floor, walls, ceiling, sci-fi ceiling grid, server towers, tech panels, framed artworks
B. Desk and PC (<code>drawDeskAndPC()</code>)	Desk, monitors, keyboard, mouse, transparent PC case, VR headset, plant, cup
C. Indoor Particle (<code>drawRoomDust()</code>)	700 floating pink dust particles

A. *Room Shell and Static Environment* (`drawRoom()`): The room uses a dark floor, three walls, and a ceiling (about $5 \times 5 \times 5$). Textures include:

- a dark marble floor texture (`tex.floor`)
- wall textures (`tex.wall` and `tex.boxSide`) for the front and side walls

Two server towers and a holographic wall panel are added, and `drawSciFiCeiling()` draws a cyan grid on the ceiling. `drawBox()` provides the frame outline, and `tex.frameLeft` is placed as the artwork.

B. *Desk and Interaction Core Area* (`drawDeskAndPC()`): A dark-gray table holds

- monitors (`tex.screen`, `tex.screen2`),
- keyboard (`tex.keyboard`),
- mouse (`drawMouseOnDesk()`),
- VR headset (`drawVRHeadset()`)
- Succulent plant (`drawSucculent()`)
- Cup (`drawCup()`)

The most recognizable element is the transparent PC case: I draws two internal fans (wire torus + small cubes), and its exterior is drawn with blending and disabled depth writing to remain semi-transparent. A small platform and a mini `drawDrone()` are placed on top.

C. *Indoor Dust Particles* (`drawRoomDust()`): The function generates 700 particles with random positions. A time-based `sinf` offset produces slow vertical floating motion.

3) *Interaction in the Room Scene*: After entering `STATE_ROOM`, a transparent HUD shows:

- [Mouse] Look Around
- [SPACE] Enter Cyber World

Pressing `SPACE` switches to `STATE_SCREEN_FLASH`, and the computer screen changes texture in preparation for the next transition, as illustrated in Fig. 4.

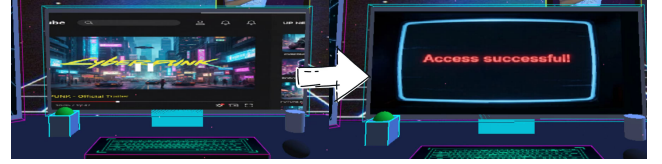


Fig. 4. Transition sequence inside the Room scene, from HUD hints to screen change and vortex entry.

D. Scene 3: Vortex Rotation Texture Transition (Transition Interface 2)

This scene provides the transition from the Room to the Cyber City. A square vortex texture is rotated around its center by translating it to the middle of the screen, applying a rotation θ , and translating it back. `GL_BLEND` is enabled to create fade-in and fade-out effects. The combined effect of the screen flash and vortex is also visible in Fig. 2.

E. Cyberpunk City (Main Interface 2)

The Cyber scene is the most complex part of the project. Its design includes the view system, world structure, city elements, traffic, weather and particles, the main character, mounts, shadows, and interaction, as illustrated in Fig. 5. Then I will show these system one by one.

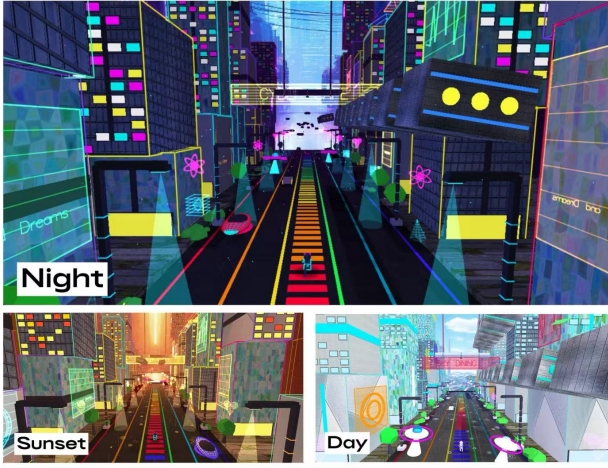


Fig. 5. Overview of the Cyberpunk City main world in the different time.

1) View and World Structure:

(1) *Third-Person Follow Camera*: A third-person camera follows the cat:

- `camDist` controls camera distance,
- `cyberCamYaw/cyberCamPitch` control rotation,
- `Z/X` adjust camera zoom.

(2) *Looped City World*: The world uses `worldOffsetZ` and `LOOP_LEN`:

- City segments repeat along the z-axis,
- Each segment contains roads, tracks, buildings, and billboards,
- Segments outside the visible range are culled.

(3) *Time-Based Lighting and Atmosphere*: `setupCyberLighting()` adjusts lighting, fog, and clear color based on `currentTimeOfDay`. Table IV summarizes the three modes.

TABLE IV
LIGHTING AND ATMOSPHERE SETTINGS

Mode	Lighting & Fog	Skybox / Texture
TIME_DAY	GL_LIGHT1 sunlight, light fog	Blue palette, <code>tex.bgDay</code>
TIME_SUNSET	Low-angle LIGHT1, orange diffuse; LIGHT2 lamps	Warm palette, <code>tex.bgSunset</code>
TIME_NIGHT	Moonlight LIGHT1; moving LIGHT2; extra LIGHT3; purple fog	Neon palette, <code>tex.bgNight</code>

2) *City Segmentation System*: The city is generated by repeating segments along the z-axis.

(1) Road and Platforms:

- `tex.road` for the main road with rainbow decorations,
- `tex.cyberPavement` for platforms and bases,
- `isTransparentFloor` enables a cyan glass-floor effect.

(2) *Multi-Layer Neon Buildings*: Buildings are stacked rectangular bodies with variations. Floors may include:

- billboard screens,
- rotating top structures and glowing rings,
- colored windows and line decorations,
- a bright ground-floor entrance.

(3) *Holographic Advertisement System*: Four types of ads are used (Table V). A combined example of building-side ads and nearby vegetation is shown in Fig. 6. Ads are drawn with `drawCyberScreen()` or `drawHologram()`, using translation, rotation, scrolling lines, or rotating 3D shapes.

TABLE V
HOLOGRAPHIC ADVERTISEMENT TYPES

Type	Position / Form
Wall Screen Ads	Mounted on building walls
Cross-Street Banner	Banner spanning the street
Rooftop Ads	Billboard on rooftops
Street Level Ads	Holograms at ground level



Fig. 6. Example of cyber trees and nearby holographic advertisements in a city segment.

(4) *Street Lamp System*: Lamps placed at `lampX` positions are built with `drawRichLamp()`, with a glowing cone representing the light area, as illustrated in Fig. 7.

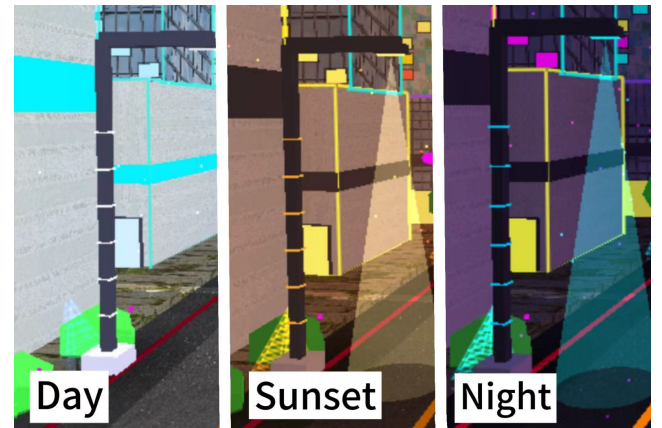


Fig. 7. Street lamp design with extended arm geometry and glowing cone light region.

(5) *Cyber Vegetation*: At each greenX position, the scene places a cyber tree (`drawCyberTree`) or bushes (`drawCyberBush`). The trunk is a cone, the crown is a rotatable polyhedron, and small spheres act as berries. An example layout with both trees and ads is shown in Fig. 6.

(6) *Mega Pipes*: In the upper layer of the Cyber City, a series of large pipes span across the streets. The design is inspired by Chongqing’s trains that pass directly through buildings. Each pipe consists of a cylindrical main body with connector rings, bottom-mounted tech boxes, and a few suspension cables added along its length. At night, the pipes display energy pulses: circular pulses travel rapidly along the surface and leave a short fading trail and additive blending is used, as illustrated in Fig. 8.

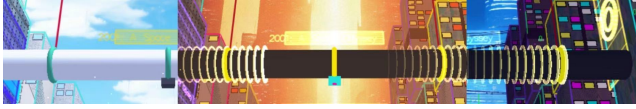


Fig. 8. Mega Pipes spanning above the Cyber City, with nighttime energy pulse effects.

3) *Traffic System*: The Cyber traffic system includes ground, aerial, and underground layers. All motion is driven by `realTime` and the world offset `worldOffsetZ`, allowing vehicles and objects to loop continuously while distant elements are culled for performance. An overview is shown in Fig. 9.

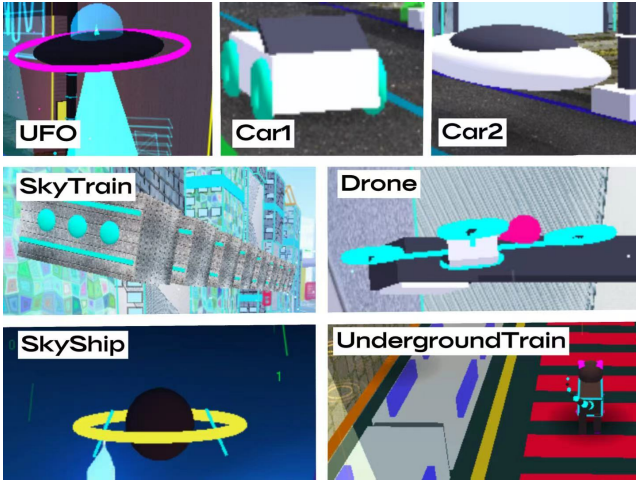


Fig. 9. Multi-layer transport system including ground vehicles, elevated trains, UFOs, drones, and underground Shinkansen.

(1) *Ground and Low-Altitude Traffic*: Hover cars (`drawHoverCar()`) and ground cars (`drawGroundCarFuture()`) run in four lanes. Each lane is defined in `drawTrafficSystem()` by its center position (`laneCenters[]`), vehicle type (`isHoverType`), direction, base speed, and color index.

(2) *Elevated Trains and Underground Shinkansen*: Sky trains use a `TrainConfig` (position, height, speed, direc-

tion, offset, theme). Train cars (`drawTrainCar()`) use `tex.skyTrain`, with window and stripe colors adjusted by time of day. Each train includes slight sway and projected shadows for depth. Two Shinkansen trains run underground in opposite directions with higher speed, longer loop cycles, and a fixed low Y position.

(3) *UFO Fleet*: UFOs enhance the vertical layers of the scene. Bodies use silver (day) or dark gray (night), with semi-transparent cockpits, a glowing ring, and a thruster cone rendered with additive blending. Formations include a figure-8 unit with gentle bobbing, a large cruiser orbiting the city, high-altitude fleets staggered along the z-axis, and low-altitude patrol units aligned with traffic lanes.

(4) *Drones and High-Tech Traffic Ships*: Eight drones move based on time t , combining circular X/Y motion with looping Z displacement using `worldOffsetZ` and `LOOP_LEN`. Bodies are drawn with `drawDrone()`.

Traffic ships (`drawTrafficShip()`) use an ellipsoid body, a rotating neon ring, and simplified line-based wings.

(5) *Daytime-Only Paper Plane Group*: During `TIME_DAY`, about 15 paper planes fly along smooth `cos/sin` curves. They are simple triangular faces drawn without lighting, using light pastel colors to create a soft daytime effect.

4) Rain, Ripples, and Particle System:

(1) *Rain and “Digital Rain”*: `isRaining` controls rain activation. `initRain()` assigns random position (x, y, z), fall speed, type, and a random offset. Heights are looped with `fmod` so raindrops recycle within a fixed range.

In `drawRainSystem()`:

Day / Sunset:

- Rain is rendered as long thin streaks using `GL_LINES`.
- Day uses cool bluish tones; sunset uses warmer orange tones.
- Blending uses `GL_SRC_ALPHA` and `GL_ONE_MINUS_SRC_ALPHA`.

Night:

- Raindrops split into line streaks and green “0/1” characters (`glutBitmapCharacter`).
- Character alpha oscillates over time, producing a slight flicker.

The overall visual effect of rain and digital rain is shown in Fig. 10.



Fig. 10. Rain and digital-rain effect rendered in the Cyber City scene.

(2) *Ground Ripples*: `drawRipples()` generates ripples with random positions offset by `worldOffsetZ`. Each ripple stores a lifespan and maximum radius and fades gradually.

Ripples are removed when expired or outside the culling range. Lighting is disabled, and additive blending creates a soft glowing outline, especially visible at night.

5) Main Character and Mounts:

(1) *Cyber Cube Cat*: The main character is a cube-style cat built from simple geometric primitives. A thin wireframe outline is added, and its color scheme switches with the time of day.

(2) *Leg and Tail Animation*: Leg motion is driven by `globalTimeCat` using sine functions. Front and back legs use a phase offset to create alternating steps. In Cloud Ride and UFO modes, the motion amplitude is reduced to maintain a stable flight pose.

The tail consists of multiple spheres. Each segment applies time-based rotation with an index offset, forming a smooth curve:

- slow left–right sway during walking,
- increased frequency when the cat accelerates.

(3) *Three Movement Modes and Mounts*: The cat switches between three states, as shown in Fig. 11.

Walking Mode

Default mode. The cat remains on the ground and moves with the camera and `worldOffsetZ`.

Cloud Ride Mode

When `isCloudMode` is true:

- the cat rises to mid-height and gently floats,
- a mechanical cloud with breathing motion, neon ring, and thruster cone is drawn below,
- the cat is placed in a riding pose for elevated exploration.

Hyper Warp Mode (UFO Mount)

When `isHyperWarp` is true:

- the cat shrinks and moves into an enlarged UFO,
- the UFO's chamber, rotation, and thruster beams are drawn first,
- the cat is positioned inside the cockpit and scaled accordingly.

All three modes reuse the same cat model, modifying only height and attachment.



Fig. 11. Cyber cube cat main character used in the Cyber City scene.

6) Details: Environmental Particles and Shadows:

(1) *Floating Particles* (`drawFloatingParticles`): Numerous particles simulate floating dust or light specks. Their z-movement uses `realTime` and random speeds, looping with modulo so particles persist without regeneration. Colors follow day, sunset, and night palettes, and additive blending produces a glowing effect.

In `isHyperWarp` mode, particles switch from `GL_POINTS` to short `GL_LINES`, creating warp-speed streaks.

(2) *Simplified Shadow System*: Two non-real-time shadow types are used:

`drawShadowBlob()` — A flattened circular fan with darker center and lighter edges. It uses `glBlendFunc(GL_DST_COLOR, GL_ZERO)` so the ground is darkened by multiplication.

`drawSquareShadow()` — A rectangular shadow for buildings and trains, drawn with slight size expansion and the same multiplicative blending.

7) *Interaction Design (HUD and Controls)*: A welcome prompt appears for about 2.5 seconds upon entering the Cyber world. Afterward, `drawHUD()` renders the control menu on the right side and updates it based on current states.

Main interactions:

- W / S: move forward / backward along the main road (controlling city flow via `worldOffsetZ`)
- Mouse movement: control camera yaw/pitch for free viewing
- Z / X: zoom camera in/out
- 1 / 2 / 3: switch time and lighting mode (DAY / SUNSET / NIGHT)
- SPACE: toggle Cloud Ride mode (`isCloudMode = !isCloudMode`)
- H / h: toggle Hyper Warp mode (`isHyperWarp = !isHyperWarp`)
- R / r: toggle the rain system (`isRaining = !isRaining`)
- T / t: toggle `isTransparentFloor` between normal road and glass-floor mode
- C / c: toggle HUD visibility (`showCyberHUD = !showCyberHUD`)
- ESC: call `exit(0)` to immediately exit the program

IV. CONCLUSION

This project builds a compact interactive 3D world by combining narrative transitions with real-time rendering. The Room scene, vortex transition, and Cyber City are integrated through an FSM structure, enabling smooth scene switching and consistent user interaction. The looped-city system, time-of-day lighting, weather and particle effects, and layered traffic contribute to a dynamic cyberpunk environment with controlled computational cost. The cube-cat character and its mount modes provide varied viewpoints and enhance user engagement.